Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Выскворкин А. В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 06.11.26

Москва, 2026

Постановка задачи

Вариант 7.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

В файле записаны команды вида: «число число число<endline>». Дочерний процесс считает их сумму и выводит результат в стандартный поток вывода. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- ssize_t write(int fd, const void* buf, size_t count); – записывает count байт из buf в файл с дескриптором fd.
- ssize_t read(int fd, void* buf, size_t count); – читает count байт из файла с дескриптором fd в buf.
- int open(const char* pathname, int flags, mode_t mode); – открывает файл, возвращает файловый дескриптор.
- int close(int fd); – закрывает файловый дескриптор.
- pid_t fork(void); – создаёт дочерний процесс.
- int execv(const char* path, char* const argv[]); – заменяет текущий образ процесса новым по пути path, передавая в него аргументы argv.
- pid_t waitpid(pid_t pid, int *stat_loc, int options); – ожидание завершения дочернего процесса.
- void exit(int status); – завершает процесс с кодом возврата status.
- int shm_open(const char *name, int oflag, mode_t mode); – создаёт и открывает или открывает существующий объект разделяемой памяти с именем *name. Возвращает файловый дескриптор для объекта разделяемой памяти.
- int shm_unlink(const char* name); – удаляет имя объекта размеляемой памяти.
- int ftruncate(int fd, off_t length); – устанавливает размер файла с дескриптором fd равным length байт.
- void *mmap(void *addr, size_t len, int prot, int flags, int fd, off_t offset); – отображает файл с дескриптором fd в адресное пространство процесса.
- sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value); – создаёт семафор или открывает существующий, value устанавливает начальное значение счётчика.
- int sem_wait(sem_t *sem); – декрементирует значение семафора. Если значение равно 0, процесс блокируется.
- int sem_post(sem_t *sem); – инкрементирует значение семафора.
- int sem_unlink(const char *name); – удаляет имя семафора.
- int sem_close(sem_t *sem); – закрывает дескриптор семафора в текущем процессе.
- int munmap(void *addr, size_t len); – убирает отображение памяти в адресное пространство процесса.

Parent создаёт объект разделяемой памяти на 4096 байт и два семафора: sem_put с начальным значением счётчика 1 и sem_take с начальным значением счётчика 0. Parent получает имя файла через stdin, открывает этот файл для чтения и создаёт дочерний процесс. Parent читает данные из файла, декрементирует семафор sem_put и записывает прочитанное в разделяемую память. В первые 6 байт записывается длина прочитанных данных, в остальные – сами данные. После этого родительский процесс инкрементирует sem_take. Дочерний процесс декрементирует sem_take, забирает данные и инкрементирует sem_put. Теперь Parent может поместить в разделяемую память следующую часть данных из файла. Child копирует данные из разделяемой памяти в свой буфер, вычисляет сумму чисел на одной строке и выводит сумму в stdout. Если Parent дочитал файл, он записывает в начало разделяемой памяти число UINT32_MAX, прочитав которое Child также завершает цикл работы, обрабатывая остаток в своём буфере.

Код программы

parent.c

```
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stdint.h>
#include <semaphore.h>
#include <stdbool.h>
#include <sys/mman.h>
#include <string.h>

#define SHM_SIZE 4096

const char SHM_NAME[] = "SharedMemory";

const char SEM_PUT_NAME[] = "CanPutInSHM";
const char SEM_TAKE_NAME[] = "CanTakeFromSHM";

int main(void) {
    int shm = shm_open(SHM_NAME, O_RDWR | O_CREAT | O_TRUNC, 0600);
    if (shm == -1) {
        const char msg[] = "error: failed to create SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    if (ftruncate(shm, SHM_SIZE) == -1) {
        const char msg[] = "error: failed to resize SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    char *shm_buf = mmap(NULL, SHM_SIZE, PROT_WRITE, MAP_SHARED, shm, 0);
    if (shm_buf == MAP_FAILED) {
        const char msg[] = "error: failed to map SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
```

```
sem_t *sem_put = sem_open(SEM_PUT_NAME, O_RDWR | O_CREAT, 0600, 1);
sem_t *sem_take = sem_open(SEM_TAKE_NAME, O_RDWR | O_CREAT, 0600, 0);
if (sem_put == SEM_FAILED || sem_take == SEM_FAILED) {
    const char msg[] = "error: failed to create semaphore\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

char filename[1024];
const char msg[] = "Enter filename: ";
write(STDOUT_FILENO, msg, sizeof(msg) - 1);

ssize_t len = read(STDIN_FILENO, filename, sizeof(filename) - 1);
if (len < 0) {
    const char msg[] = "error: failed to read filename\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}
filename[len] = '\0';
if (len > 0 && filename[len - 1] == '\n')
    filename[len - 1] = '\0';

int file = open(filename, O_RDONLY);
if (file == -1) {
    const char msg[] = "error: failed to open file\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

const pid_t child = fork();
if (child == 0) {
    char *const args[] = {"./child", NULL};
    int status = execv("./child", args);
    if (status == -1) {
        const char msg[] = "error: failed to exec\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
} else if (child == -1) {
    const char msg[] = "error: failed to spawn new process\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

bool running = true;
while (running) {
    char buf[SHM_SIZE - sizeof(uint32_t)];
    ssize_t bytes = read(file, buf, sizeof(buf));
    if (bytes == -1) {
        const char msg[] = "error: failed to read from file\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    sem_wait(sem_put);
```

```
        uint32_t *length = (uint32_t *)shm_buf;
        char *text = shm_buf + sizeof(uint32_t);

        if (bytes > 0) {
            *length = (uint32_t)bytes;
            memcpy(text, buf, (size_t)bytes);
        } else {
            *length = UINT32_MAX;
            running = false;
        }

        sem_post(sem_take);
    }

    close(file);

    waitpid(child, NULL, 0);

    sem_unlink(SEM_PUT_NAME);
    sem_unlink(SEM_TAKE_NAME);
    sem_close(sem_put);
    sem_close(sem_take);
    munmap(shm_buf, SHM_SIZE);
    shm_unlink(SHM_NAME);
    close(shm);

    return 0;
}
```

child.c

```
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <fcntl.h>
#include <stdint.h>
#include <semaphore.h>
#include <stdbool.h>
#include <sys/mman.h>

#define SHM_SIZE 4096

const char SHM_NAME[] = "SharedMemory";

const char SEM_PUT_NAME[] = "CanPutInSHM";
const char SEM_TAKE_NAME[] = "CanTakeFromSHM";

void print_sum(char* buf, size_t len) {
    buf[len] = '\0';

    char* p = buf;
    float sum = 0.0f;

    while(*p != '\0') {
        char *end;
```

```
        float val = strtof(p, &end);

        if (end == p)
            p++;
        else {
            sum += val;
            p = end;
        }
    }

    char ans[64];
    int n = snprintf(ans, sizeof(ans), "%f\n", sum);
    if (n < 0) {
        const char msg[] = "error: failed to print sum\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    write(STDOUT_FILENO, ans, n);
}

int main(int argc, char **argv) {

    int shm = shm_open(SHM_NAME, O_RDWR, 0);
    if (shm == -1) {
        const char msg[] = "error: failed to open SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    char *shm_buf = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm, 0);
    if (shm_buf == MAP_FAILED) {
        const char msg[] = "error: failed to map SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    sem_t *sem_put = sem_open(SEM_PUT_NAME, O_RDWR);
    sem_t *sem_take = sem_open(SEM_TAKE_NAME, O_RDWR);
    if (sem_put == SEM_FAILED || sem_take == SEM_FAILED) {
        const char msg[] = "error: failed to open semaphore\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    char buf[SHM_SIZE];
    size_t i = 0;

    bool running = true;
    while (running) {
        sem_wait(sem_take);

        uint32_t *length = (uint32_t *)shm_buf;
        const char *text = shm_buf + sizeof(uint32_t);
        uint32_t cur_len = *length;

        if (cur_len == UINT32_MAX)
```

```
            running = false;
        else if (cur_len > 0) {
            if (i + cur_len <= sizeof(buf)) {
                memcpy(buf + i, text, cur_len);
                i += cur_len;
            } else {
                sem_post(sem_put);
                const char msg[] = "error: sum buffer overflow\n";
                write(STDERR_FILENO, msg, sizeof(msg) - 1);
                exit(EXIT_FAILURE);
            }

        }

        sem_post(sem_put);

        size_t start = 0;
        for (size_t j = 0; j < i; j++) {
            if (buf[j] == '\n') {
                print_sum(buf + start, j - start);
                start = j + 1;
            }
        }

        if (start > 0) {
            size_t unfin = i - start;
            if (unfin > 0)
                memmove(buf, buf + start, unfin);
            i = unfin;
        }
    }

    if (i > 0) {
        print_sum(buf, i);
        i = 0;
    }

    sem_close(sem_put);
    sem_close(sem_take);
    munmap(shm_buf, SHM_SIZE);
    close(shm);

    return 0;
}
```

Протокол работы программы

TestOne

1.23 4.48 25.17

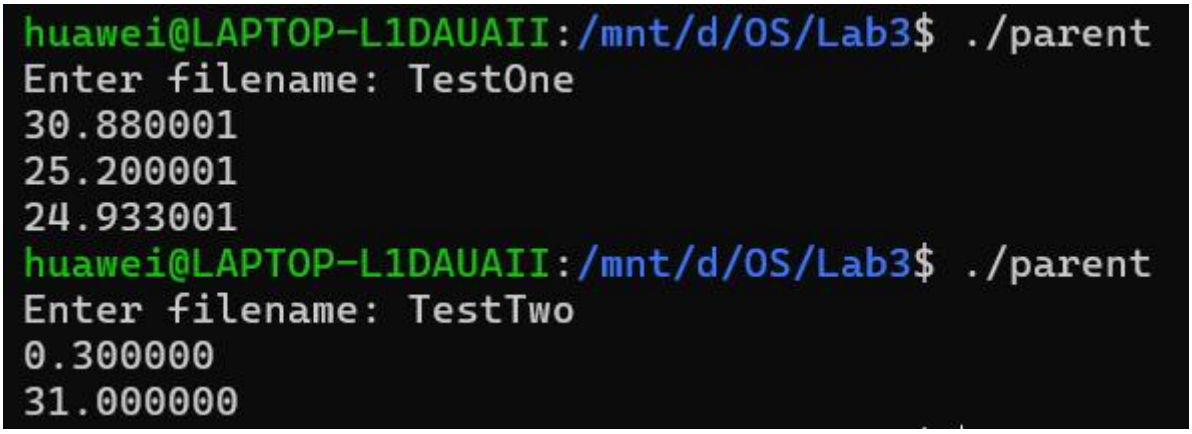
15.2 10

3.333 9 12.6

TestTwo

0.1 0.2

15 2 2 4 8



Вывод

В результате выполнения лабораторной работы были приобретены практические навыки в обеспечении обмена данными между процессами посредством разделяемой памяти и технологии “File mapping”. Освоены принципы работы семафора для синхронизации работы процессов с целью избежания гонки данных. Закреплены навыки работы с файловыми системами и механизмами создания дочерних процессов в UNIX-подобных системах.